The Journal of Open Source Software

dfreproject: A Python package for astronomical reprojection

Carter Lee Rhea[1,2*], **Pieter Van Dokkum**[1,3*], **Steven R. Janssens**[1], **Imad Pasha**[1,3], **Roberto Abraham**[1,4,5], **William P Bowman**[1,3], **Deborah Lokhorst**[1,6], **and Seery Chen**[1,4,5]

1 Dragonfly Focused Research Organization, 150 Washington Avenue, Santa Fe, 87501, NM, USA **2** Centre de recherche en astrophysique du Québec (CRAQ) **3** Astronomy Department, Yale University, 219 Prospect St, New Haven, CT 06511, USA **4** David A. Dunlap Department of Astronomy & Astrophysics, University of Toronto, 50 St. George Street, Toronto, ON M5S3H4, Canada **5** Dunlap Institute for Astronomy & Astrophysics, University of Toronto, 50 St. George Street, Toronto, ON M5S3H4, Canada **6** NRC Herzberg Astronomy & Astrophysics Research Centre, 5071 West Saanich Road, Victoria, BC V9E 2E7, Canada * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 07 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

1. Summary

Deep astronomical images are often constructed by digitially stacking many individual sub-exposures. Each sub-exposure is expected to show small differences in the positions of stars and other objects in the field, due to the movement of the celestial bodies, changes/imperfections in the opto-mechanical imaging train, and other factors. To maximize image quality, one must ensure that each sub-exposure is aligned to a common frame of reference prior to stacking. This is done by reprojecting each exposure onto a common target grid defined using a World Coordinate System (WCS) that is defined by mapping the known angular positions of reference objects to their observed spatial positions on each image. The transformations needed to reproject images involve complicated trigonometric expressions which can be slow to compute, so reprojection can be a major bottleneck in image processing pipelines.

To make astronomical reprojections faster to implement in pipelines, we have written `dfreproject`, a Python package of GPU-optimized functions for this purpose. The package's functions break down coordinate transformations using Gnomonic projections to define pixel-by-pixel shifts from the source to the target plane. The package also provides tools for interpolating a source image onto a target plane with a single function call. This module follows the FITS and SIP formats laid out in the following papers: Greisen & Calabretta (2002), Calabretta & Greisen (2002), and Shupe et al. (2005). Compared to common alternatives, `dfreproject`'s routines result in speedups of up to 20X when run on a GPU and 10X when run on a CPU.

2. Statement of need

Several packages already exist for calculating and applying the reprojection of a source image onto a target plane, such as `reproject` (Robitaille et al., 2020) or `astroalign` (Beroiz et al., 2020). These packages excel at easy-to-use, general-purpose astronomical image reprojection, but they function solely on CPUs and can be computational bottlenecks in some data reduction pipelines. The `dfreproject` package harnesses GPUs (using `PyTorch` (Paszke et al., 2019) as its computational backbone) to improve computational efficiency. The package has minimal reliance on pre-existing WCS packages such as those found in `astropy` or `WCSLIB` (Astropy Collaboration et al., 2013, 2018, 2022), with such packages being used only for convenience in

non-computationally expensive steps (such as using `astropy.wcs` to read the header information from the input fits files).

`dfreproject`'s primary purpose is to reproject observations taken by a new version of the Dragonfly Telephoto Array that is currently under construction in Chile. The volume of data obtained with this telescope will be large, with >1000 exposures obtained simultaneously, and it is paramount that the processing pipeline incorporates fast and accurate reprojection methods. As shown below, by leveraging `PyTorch` for vectorization and parallelization via the GPU, `dfreproject` achieves a considerable speedup (up to nearly 20X) over standard methods.

`dfreproject` can be used as a direct replacement for `reproject.reproject_interp` by simply importing `dfreproject` instead of `reproject` such as:

```
from dfreproject import calculate_reprojection
reprojected = calculate_reprojection(
    source_hdus=source_hdu,
    target_wcs=target_wcs,
    shape_out=target_hdu.data.shape,
    order='bilinear'
)
```

The `target_wcs` argument can be passed as a header similar to `reproject`. Additionally, if `shape_out` is not provided, the shape will be the same as the input.

3. Methods

3.1 Overview

We must perform three intermediate calculations to reproject an image onto a new coordinate plane. To do this, we use the target and source WCS.

Before defining the steps, there are a few terms to define:

- SIP: Standard Imaging Polynomial. This convention allows us to represent non-linear geometric distortions as a simple polynomial. The order and coefficients of this polynomial are stored in the header. The SIP is broken down into four individual polynomials, SIP_A, SIP_B, SIP_INV_A, and SIP_INV_B where SIP_A defines the polynomial applied to the x-coordinates, SIP_B defines the polynomial applied to the y-coordinates, and SIP_INV_A and SIP_INV_B define the inverse operations. For an in-depth discussion on SIP, please see Shupe et al. (2005).

CD Matrix: Coordinate Description Matrix. This is a 2X2 matrix that encodes the image's rotation, skew, and scaling. The values are conveniently stored in the header. The CD matrix may also be constructed from the PC Projection Coordinate matrix multiplied by the CDELT values.

The steps are as follows:

1. For each pixel, calculate the corresponding celestial coordinate using the target WCS
    1. Apply shift
    2. Apply SIP distortion
    3. Apply CD matrix
    4. Apply transformation to celestial coordinates using the Gnomonic projection
2. Calculate the position in the source grid for each celestial coordinate. This provides the offset for the next step.
    1. Apply the inverse transformation using the Gnomonic projection

DRAFT

    2. Apply inverse CD matrix
    3. Apply inverse SIP distortion
    4. Apply shift
3. Interpolate the source image onto the newly calculated grid

In the final interpolation step, we include local flux conservation by simultaneously projecting an identity tensor called the footprint. The final reprojected frame is normalized by this footprint.

3.2 Coordinate Transformation

In this section, we describe the coordinate transformation using the Gnomonic projection. Please note that we include an additional shift of 1 pixel to handle Python being 0-based. We will be using the following definitions for values:

x, y - pixel values

$\mathrm{crpix}_1, \mathrm{crpix}_2$ - center pixels as defined in WCS

$\mathrm{dec}_0, \mathrm{ra}_0$ - Central Declination and Right Ascension as defined in the WCS.

All trigonometric functions require the values to be in radians.

3.2.1 To celestial coordinates

For these calculations, we use the WCS information for the target plane.

$$u = x - (\mathrm{crpix}_1 - 1)$$
$$v = y - (\mathrm{crpix}_2 - 1)$$

$$u = u - \mathrm{SIP_A}(u, v)$$
$$v = v - \mathrm{SIP_B}(u, v)$$

$$\begin{bmatrix} u' \\ v' \end{bmatrix} = \mathrm{CD} \begin{bmatrix} u \\ v \end{bmatrix}$$
$$r = \sqrt{u'^2 + v'^2}$$
$$r_0 = \frac{180}{\pi}$$
$$\phi = \mathrm{atan}^2(-u', v')$$
$$\theta = \mathrm{atan}^2(r_0, r)$$
$$\mathrm{dec} = \sin^{-1}\Big(\sin(\theta)\sin(dec_0) + \cos(\theta)\cos(dec_0)\cos(\phi)\Big)$$
$$\mathrm{ra} = \mathrm{ra}_0 + \mathrm{atan}^2\Big(-\cos(\theta)\sin(\phi), \sin(\theta)\cos(dec_0) - \cos(\theta)\sin(dec_0)\cos(\phi)\Big)$$

DRAFT

3.2.2 To source coordinates

For these calculations, we use the WCS information for the source plane.

$$\Delta\mathrm{ra} = \mathrm{ra} - \mathrm{ra}_0$$

$$\phi = \mathrm{atan}^2\Big(-\cos(\mathrm{dec})\sin(\Delta\mathrm{ra}), \sin(\mathrm{dec})\cos(\mathrm{dec}_0) - \cos(\mathrm{dec})\sin(\mathrm{dec}_0)\cos(\Delta\mathrm{ra})\Big)$$

$$\theta = \sin^{-1}\Big(\sin(\mathrm{dec})\sin(\mathrm{dec}_0) + \cos(\mathrm{dec})\cos(\mathrm{dec}_0)\cos(\Delta\mathrm{ra})\Big)$$

$$r = r_0 \frac{\cos(\theta)}{\sin(\theta)}$$

$$u' = -r\sin(\phi)$$

$$v' = r\cos(\phi)$$

$$\begin{bmatrix} u \\ v \end{bmatrix} = \mathrm{CD_INV} \begin{bmatrix} u' \\ v' \end{bmatrix}$$

$$u = u - \mathrm{SIP_INV_A}(u, v)$$

$$v = v - \mathrm{SIP_INV_B}(u, v)$$

$$x = u + (\mathrm{crpix}_1 - 1)$$

$$y = v + (\mathrm{crpix}_2 - 1)$$

4. Results

4.1 Demo

We created two small (50x50) FITS files for this demonstration with a pixel offset of 0.5 pixels and a 0.005 degree offset in the heeader. In Figure 1 from left to right, we show the original image, the `dfreproject` solution, the `reproject` solution, and the relative error between the two. We define the relative error as $100 * \frac{\mathrm{dfreproject_solution} - \mathrm{reproject_solution}}{\mathrm{reproject_solution}}$. For both solutions, we use a bilinear interpolation scheme. In the noisy regions of the image, the differences in the reprojections is pure noise. There are slight differences in the solutions at the locations of the Gaussians which is attributable to small differences in the normalization.

Figure 1

4.2 Speed Comparison

To compare the execution times, we created a benchmarking script (which can be found in the demos/benchmarking directory under `benchmark-script.py`; the figures are constructed with `benchmark-plotter.py`). This test is run between `dfreproject` and `reproject`. We benchmark the three interpolation schemes with and without SIP distortion for images sized 256x256 to 4000x6000 (this approximately matches the size of Dragonfly images). Figure 2 shows the results of this benchmarking when `dfreproject` is run using a GPU (NVIDIA GeForce RTX 4060).

DRAFT

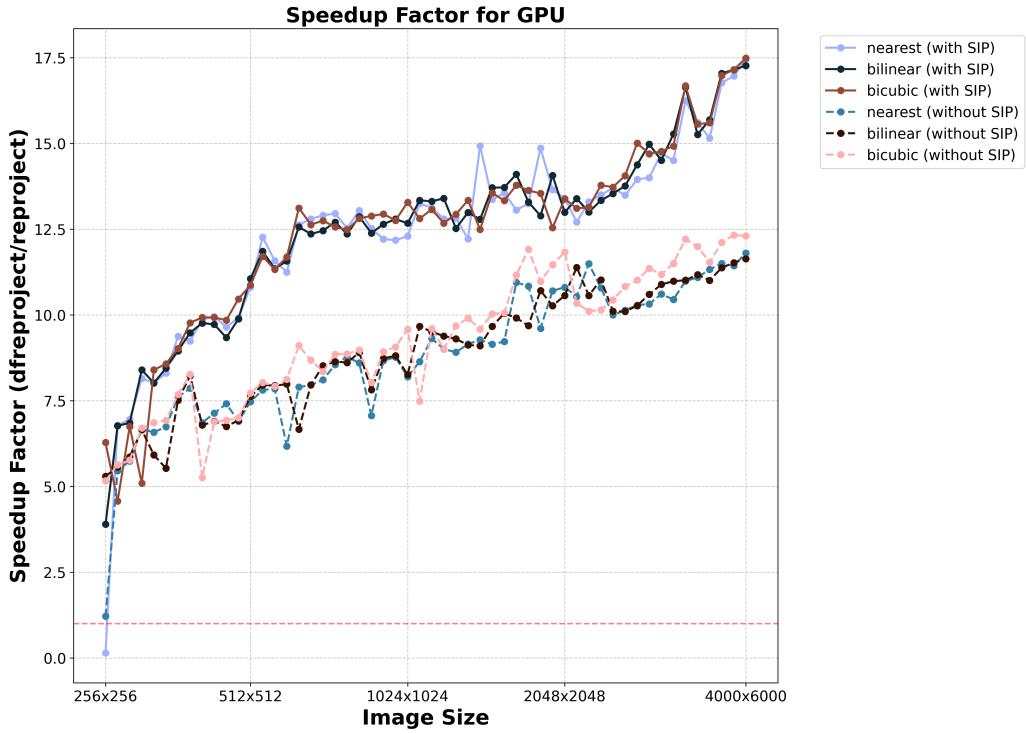

Figure 2

As evidenced by this figure, `dfreproject` has a significant speed advantage over `reproject` for larger images regardless of the type of interpolation scheme. The speedup is most pronounced in the case where SIP distortions are included.

In Figure 3, we display the same results except we used a CPU (Intel® Core™ i9-14900HX).

DRAFT

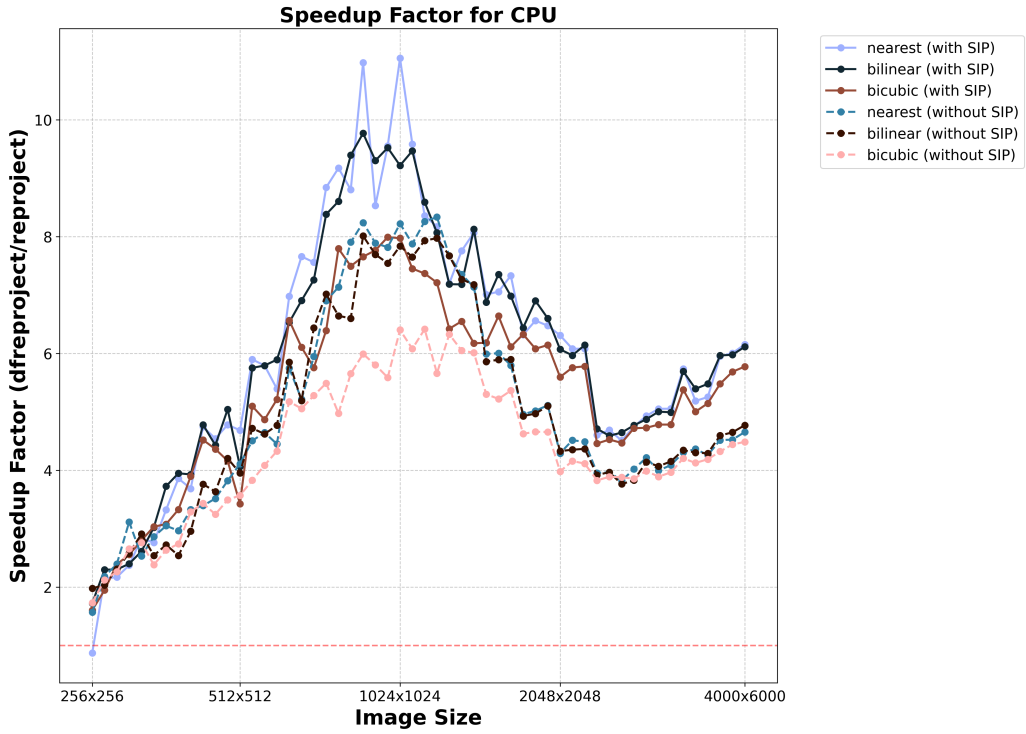

Figure 3

Although the speedup on the CPU is not as impressive as on the GPU, it is still considerable. All code can be found in the demo directory.

Acknowledgements

We acknowledge the Dragonfly FRO and particularly thank Lisa Sloan for her project management skills.

We use the cmcrameri scientific color maps in our demos (Crameri, 2023). # References

Astropy Collaboration, Price-Whelan, A. M., Lim, P. L., Earl, N., Starkman, N., Bradley, L., Shupe, D. L., Patil, A. A., Corrales, L., Brasseur, C. E., Nöthe, M., Donath, A., Tollerud, E., Morris, B. M., Ginsburg, A., Vaher, E., Weaver, B. A., Tocknell, J., Jamieson, W., … Astropy Project Contributors. (2022). The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package. *The Astrophysical Journal*, *935*, 167. https://doi.org/10.3847/1538-4357/ac7c74

Astropy Collaboration, Price-Whelan, A. M., Sipőcz, B. M., Günther, H. M., Lim, P. L., Crawford, S. M., Conseil, S., Shupe, D. L., Craig, M. W., Dencheva, N., Ginsburg, A., VanderPlas, J. T., Bradley, L. D., Pérez-Suárez, D., Val-Borro, M. de, Aldcroft, T. L., Cruz, K. L., Robitaille, T. P., Tollerud, E. J., … Astropy Contributors. (2018). The Astropy Project: Building an Open-science Project and Status of the v2.0 Core Package. *The Astronomical Journal*, *156*, 123. https://doi.org/10.3847/1538-3881/aabc4f

Astropy Collaboration, Robitaille, T. P., Tollerud, E. J., Greenfield, P., Droettboom, M., Bray, E., Aldcroft, T., Davis, M., Ginsburg, A., Price-Whelan, A. M., Kerzendorf, W. E., Conley, A., Crighton, N., Barbary, K., Muna, D., Ferguson, H., Grollier, F., Parikh, M. M., Nair, P. H., … Streicher, O. (2013). Astropy: A community Python package for astronomy. *Astronomy and Astrophysics*, *558*, A33. https://doi.org/10.1051/0004-6361/201322068

DRAFT

Beroiz, M., Cabral, J. B., & Sanchez, B. (2020). Astroalign: A Python module for astronomical image registration. *Astronomy and Computing*, *32*, 100384. https://doi.org/10.1016/j.ascom.2020.100384

Calabretta, M. R., & Greisen, E. W. (2002). Representations of celestial coordinates in FITS. *A&A*, *395*(3), 1077–1122. https://doi.org/10.1051/0004-6361:20021327

Crameri, F. (2023). *Scientific colour maps*. Zenodo. https://doi.org/10.5281/zenodo.8409685

Greisen, E. W., & Calabretta, M. R. (2002). Representations of world coordinates in FITS. *Astronomy and Astrophysics*, *395*, 1061–1075. https://doi.org/10.1051/0004-6361:20021326

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. https://doi.org/10.48550/arXiv.1912.01703

Robitaille, T., Deil, C., & Ginsburg, A. (2020). Reproject: Python-based astronomical image reprojection. *Astrophysics Source Code Library*, ascl:2011.023. https://ui.adsabs.harvard.edu/abs/2020ascl.soft11023R

Shupe, D. L., Moshir, M., Li, J., Makovoz, D., Narron, R., & Hook, R. N. (2005). *The SIP Convention for Representing Distortion in FITS Image Headers*. *347*, 491. https://ui.adsabs.harvard.edu/abs/2005ASPC..347..491S

DRAFT